

Relazione Finale di Samuel Iskander Helal Khalil

Tecnico dello sviluppo e progettazione di programmi informatici

Anno formativo: 2025/2026

Scelte Progettuali

Ho scelto Django perché durante una o due delle lezioni in cui è stato spiegato FastAPI ho avuto l'influenza, e pur essendo stato fisicamente presente, non ho potuto prestare un livello di attenzione (in presenza o nello svolgere gli esercizi a casa) simile a quello che ho prestato durante le lezioni di Django.

Inoltre, ci sono alcuni progetti personali che sto considerando di iniziare per cui Django, essendo più "batteries-included" (grosso vantaggio per un principiante, per ragioni di difficoltà e sicurezza), è a mio parere più adatto, quindi le ricerche fatte nel mio tempo libero sono state perlopiù su di esso.

Descrizione dello Sviluppo

Setup iniziale

Boilerplate standard, che include:

- inizializzazione del progetto (comando `django-admin startproject tastydelivery .`)
- creazione delle app (comandi `python manage.py startapp gestione_ristoranti` e `python manage.py startapp gestione_ordini`)*
- creazione del database (in MySQL Workbench, con `create database 'tastydelivery'`)
- setup del file `tastydelivery\settings.py`
 - inserimento delle app in `INSTALLED_APPS`
 - sostituzione del contenuto di `DATABASES` con versione che fa affidamento alla funzione `os.getenv` (che chiaramente ho dovuto importare, e caricare con `load_dotenv()`) per permettermi di inserire questi dati nel file `.env`
- inserimento delle variabili precedentemente parte di `DATABASES` all'interno del sopracitato file `.env`
- inserimento delle path per le app `include("gestione_ristoranti.urls")` e `include("gestione_ordini.urls")` nel file `tastydelivery\urls.py` per utilizzare gli endpoint nei file `urls.py` delle app individuali, invece di quello "globale" dell'app principale.

* Ho inizialmente creato solo l'app `api` per velocizzare lo sviluppo iniziale, e successivamente ho creato le due app finali in cui ho copiato il codice (con le dovute modifiche)

Creazione dei modelli

Nei file `<nome app>\models.py` ho creato i modelli delle tabelle secondo la consegna utilizzando per ciascuna di esse una classe (che eredita da `models.Model`). All'interno di ciascuna classe ho impostato i campi utilizzando la seguente sintassi:

```
nome = models.CharField(max_length=50, null=False)
# campo chiamato "nome", di tipo varchar(50), not null

tipologia_id = models.ForeignKey(Tipologia,
on_delete=models.PROTECT,related_name="ristoranti")
# campo "tipologia_id" di tipo int, popolato dalle foreign key del modello
"Tipologia"

# Ecc...
```

Ho inoltre aggiunto il metodo `__str__` che restituisce una f string che ne combina alcuni dettagli chiave, principalmente per la visualizzazione nel pannello admin. Esempio:

```
def __str__(self):
    return f"ordine #{self.id} [{self.data_ora.date()}] - {self.ristorante_id.nome} - {self.cliente_id.nome}"
```

Infine ho creato ed applicato le migrazioni con i seguenti due comandi:

- `python manage.py makemigrations`
- `python manage.py migrate`

Creazione dei serializer

Nei file `<nome app>\serializers.py` ho impostato la serializzazione dei modelli importandoli ed assegnandoli all'attributo `model` della sottoclasse `Meta` della classe `<modello>Serializer` (che eredita dalla classe importata `serializers.ModelSerializer`)

Per i modelli che hanno campi relazionali ho assegnato all'attributo `fields` la lista dei campi uno per uno, mentre per i modelli che non li includono ho semplicemente usato `"__all__"`

Per includere la testata degli ordini nel database (come richiesto nel modulo 2) ho aggiunto l'attributo `testata = serializers.SerializerMethodField()` che mi permette di popolare il campo con un metodo nel serializer, che ho successivamente creato come segue:

```
def get_testata(self, obj):
    return f"ordine #{obj.id} [{obj.data_ora.date()}] - {obj.ristorante_id.nome} - {obj.cliente_id.nome}"
# passandogli `obj` posso leggere gli attributi della specifica istanza al momento
della creazione, ed assemblare una f string con tutte le informazioni dell'ordine,
da inserire nel campo `testata`.
```

Per il serializer del modello `OrdineDettaglio`, ho esplicitamente aggiunto i due campi a cui fa riferimento con:

```
ordine_id = OrdineSerializer(many=False, read_only=True)
# Non ho fatto la sezione bonus quindi non sono sicuro che sia corretto
```

Creazione dei viewset

Nei file `<nome app>\views.py` ho impostato le view usando la seguente sintassi:

```
class ClienteViewSet(viewsets.ModelViewSet): # <modello>ViewSet(classe
ModelViewSet precedentemente importata)
    queryset = Cliente.objects.all() # <risultato ottenuto> = funzione che va a
    prendere tutte le righe del modello Cliente (anch'esso importato sopra)
    serializer_class = ClienteSerializer
```

Per le funzioni che gestiscono le richieste GET e POST ho fatto affidamento a quelle presenti nel cheatsheet (ovviamente modificandole dove opportuno), che utilizzano i seguenti elementi:

- decoratore `@action` che stabilisce il tipo di richiesta e i suoi parametri
- la funzione `self.get_object()` che ottiene l'istanza in questione
- la funzione `request.data.get('id')` che ottiene l'ID dalla richiesta (che essendo una GET, lo trasmette tramite l'URL)
- un breve if statement per assicurarsi che un elemento con quell'ID è effettivamente presente nella tabella
- la funzione `<modello>.add(oggetto)` per le richieste POST, che permettono all'oggetto (la "riga") creato dalla GUI swagger (e dalla funzione `get_object_or_404(<modello>, pk=id_<riga modello>)`) di essere aggiunto alla tabella.

Setup della pagina /admin

nei file `<nome app>\admin.py` ho registrato i relativi modelli (per collegarli al pannello admin), prima importandoli, e successivamente usando per ciascuno di essi una riga come la seguente:

```
admin.site.register(<modello>) # necessaria importazione `from django.contrib
import admin`
```

Ho creato il superuser per accedere al pannello usando il comando `python manage.py createsuperuser` e successivamente scegliendo le seguenti credenziali:

- username: "admin"
- password: "password"
- email: ""

Ovviamente è un obbrobrio dal punto di vista della sicurezza, ma essendo questo puramente un esercizio, ho pensato di minimizzare le probabilità di perdere l'accesso, dandogli appunto le credenziali più "standard" possibili.

Creazione del mockup per il front-end

Per essere sicuro di rientrare nelle scadenze temporali ho scelto di fare due pagine molto minimali che includono i requisiti minimi e poco altro.

Ho fatto una semplice navbar scura (per contrastare con lo sfondo bianco del resto del sito), con un elemento **navbar-brand** a sinistra per il nome del sito, e la navigazione (due link alle due pagine) a destra (differenziati dalla classe active).

Per il catalogo ho semplicemente usato un elemento **form-select** con alcuni tipi di ristorante come opzione, e delle semplicissime card (con solo elementi **card-title** e **card-text**, rispettivamente per titolo e descrizione/tipologia), con soglia per il reflow settata al breakpoint medium.

Per la simulazione dell'ordine ho utilizzato:

- un elemento input testuale per il nome del cliente (ho scelto di non usare l'ID perchè mi è sembrata un'interfaccia per gli utenti, e non mi sembra sensato dare loro questa opzione).
- un elemento **form-select** per la selezione del ristorante
- un elemento **form-check** per le checkbox della scelta dei piatti
- un bottone di tipo **submit** per dare l'ok finale

Infine ho aggiunto un semplice footer con sfondo **bg-light** contenente un testo `<small>` con copyright ed email di contatto.

Database

Decisioni prese durante la creazioni dei modelli

ristorante -[many-to-one]-> tipologia (un ristorante può appartenere ad una sola tipologia, ma ogni tipologia può essere assegnata a più ristoranti) Ho deciso di usare **on_delete=models.PROTECT** perché secondo me ha senso che all'eliminazione di una tipologia non vengano anche eliminati i ristoranti che gli appartengono (una tipologia potrebbe essere eliminata per sbaglio, e se esiste ancora un ristorante nel DB non ha senso che venga eliminato per alcun motivo al di fuori della sua specifica deliberata eliminazione).

piatto -[one-to-many]-> ristorante (un piatto può appartenere ad un solo ristorante, ma un ristorante può avere più di un piatto). Ho deciso di usare **on_delete=models.CASCADE**, perché dopo l'eliminazione di un ristorante, non c'è più strettamente bisogno di conservare i suoi piatti, e quindi è secondo me accettabile cancellarli.

ordine -[many-to-one]-> cliente (un cliente può avere più ordini, ma un ordine può appartenere solo ad un cliente) **ordine -[one-to-many]-> ristorante** (ogni ordine appartiene ad un singolo ristorante, ma un ristorante può avere più ordini) Ho deciso di usare **on_delete=models.PROTECT** su entrambe perché secondo me gli ordini devono essere sempre presenti nel DB, come minimo per ragioni di contabilità, e non ha senso che vengano eliminati, e se sono presenti, è giusto che venga mantenuto il riferimento al cliente, in caso servisse risalire ad esso.

Per gli stati dell'ordine ho scelto di impostare i seguenti 4 stadi, che mi sono sembrati plausibili per un'app di questo tipo:

- In elaborazione

- In preparazione
- In transito
- Consegnato Onestamente non ne ho mai usate, quindi mi sono basato sul tracciamento dei pacchi (come ad esempio quelli di Amazon e simili).

dettagli_ordine -[one-to-one]-> ordine (ogni ordine è solo presente in un dettaglio ordine, ed ogni dettaglio ordine riguarda solo un ordine) Ho deciso di usare `on_delete=models.CASCADE` perché se si decide di eliminare esplicitamente un ordine, credo voglia dire che l'ordine è stato annullato, e quindi non è più necessario conservarne i dettagli (perché effettivamente non è mai avvenuto). Parto dal presupposto che se un ordine è effettivamente avvenuto, non verrà presa la decisione di eliminarlo (ovviamente riconosco che è molto possibile che ciò avvenga comunque per errore, e che quindi la soluzione corretta è di fare qualcosa tipo cambiare un campo "deleted" da False a True, che ho scelto di non fare per questioni di tempo).

dettagli_ordine -[one-to-many]-> piatto (un piatto può essere presente in molti dettagli ordine, ma ogni dettaglio ordine contiene un piatto solo)

Problematiche e Soluzioni

La problematica principale che ho incontrato è stata nel riadattare una funzione per la richiesta POST da un serializer ad un altro. Pur avendo soltanto modificato i nomi, per qualche motivo la seconda mostrava i campi sbagliati nella GUI swagger. La differenza si è poi rivelata essere non nella funzione stessa (che ha senso), ma nei diversi serializer che utilizzavano. Evidentemente avevo fatto un errore nell'implementazione del secondo (che doveva comunque essere diverso). Una volta modificato per renderlo più simile al primo, l'endpoint ha cominciato a mostrare i campi corretti, e l'API ha iniziato a funzionare come dovrebbe, sia nella swagger che nel pannello admin.

Un'altro problema (a mio parere minore) è la root della UI swagger, che include solo gli URL dell'app `gestione_ristoranti`, e non quelli dell'app `gestione_ordini`. Gli endpoint sono tutti presenti e funzionanti, ma per una ragione che non sono riuscito a scoprire non appaiono nella sopracitata interfaccia.

Durante la prova in generale, ma principalmente per lo sviluppo dei serializer e le view, ho fatto affidamento al cheatsheet messo a nostra disposizione dal docente. Ad essere sincero non posso dire di aver completamente memorizzato la sintassi di Django e le sue librerie (ricordarla è stata la difficoltà principale), quindi avere questo documento di riferimento, del resto molto simile a quello che avrei nel mondo reale, assieme ad un'infinità di altre risorse, è stato fondamentale per rinfrescarmi la memoria ed applicare i principi che conosco concettualmente, ma che dovrei praticare ancora un bel po' prima di poterli applicare interamente a memoria. Per fortuna negli sviluppi che farò nel mondo reale non ci sono queste restrizioni, quindi direi che se dovessi sviluppare un backend simile (o anche più complesso), grazie all'ausilio delle suddette risorse (Internet, libri, IA, ecc) ne sarei decisamente capace.